

. Librerías y lectura de la tabla

```
library(sparklyr)
library(dplyr)
library(tibble)
```

```
# =====
# READ FINAL MODEL TABLE
# =====
```

```
model_df <- sdf_sql(
  sc,
  "
  SELECT
    *,
    CAST(label_3m AS DOUBLE) AS label
  FROM bu_frd.model_features_ab_final_2022_2025
  "
)
```

```
# =====
# TEMPORAL SPLITS
# =====
```

```
train_df <- model_df %>%
  filter(model_split == "TRAIN")
```

```
validation_df <- model_df %>%
  filter(model_split == "VALIDATION")
```

```
test_df <- model_df %>%
  filter(model_split == "TEST")
```

Hagamos primero un QA:

```
model_df %>%
  group_by(model_split, label) %>%
  summarise(
    rows = n(),
    .groups = "drop"
  ) %>%
  arrange(model_split, desc(label)) %>%
  collect()
```

se espera :

TRAIN

1 27,033

0 270,330

VALIDATION

1 23,557

0 235,570

TEST

1 12,009

0 120,090

## 2. Definir Model A

```
features_A <- c(
```

```
# -----  
# Sparse behavior  
# -----
```

```
"deterioration_acceleration",  
"adverse_acceleration",  
"max_dpd_jump_recent_3m",
```

```
# -----  
# Active accounts  
# -----
```

```
"active_accounts_change_vs_baseline",  
"active_accounts_changed_flag",
```

```
# -----  
# Balance  
# -----
```

```
"balance_change_model",  
"log1p_abs_balance_change_model",
```

```
"balance_change_available_flag",  
"balance_recent_months_available",  
"balance_prior_months_available",
```

```
# -----  
# Past due  
# -----
```

```

"past_due_change_model",
"past_due_change_available_flag",
"past_due_recent_months_available",
"past_due_prior_months_available",

# -----
# Delinquency
# -----

"delinquency_share_change_model",
"delinquency_change_available_flag",
"delinquency_recent_months_available",
"delinquency_prior_months_available",

# -----
# Utilization
# -----

"avg_utilization_change_model",
"weighted_utilization_change_model",

"utilization_change_available_flag",
"utilization_recent_months_available",
"utilization_prior_months_available"
)

```

Debemos obtener:

```
length(features_A)
```

### 3. Definir Model B

```

features_B <- c(

  features_A,

  # -----
  # Selected Z-score challenger features
  # -----

  "active_accounts_z_model",
  "active_accounts_z_available_flag",

  "avg_utilization_z_model",
  "avg_utilization_z_available_flag",

  "weighted_utilization_z_model",

```

```
"weighted_utilization_z_available_flag"  
)
```

```
length(features_B)
```

#### 4. Construir las fórmulas en R

```
formula_A <- reformulate(  
  termlabels = features_A,  
  response = "label"  
)
```

```
formula_B <- reformulate(  
  termlabels = features_B,  
  response = "label"  
)
```

```
formula_A  
formula_B
```

#### 6. Función para entrenar Logistic Regression

```
fit_logistic_model <- function(  
  train_data,  
  validation_data,  
  formula,  
  reg_param  
) {  
  
  # =====  
  # FIT MODEL  
  # =====  
  
  model <- ml_logistic_regression(  
    x = train_data,  
    formula = formula,  
  
    fit_intercept = TRUE,  
  
    reg_param = reg_param,  
    elastic_net_param = 0,  
  
    max_iter = 100,  
  
    family = "binomial"  
  )
```

```
# =====
# VALIDATION PREDICTIONS
# =====

predictions <- ml_predict(
  model,
  validation_data
)

return(
  list(
    model = model,
    predictions = predictions
  )
)
}
```

## 7. Extraer la probabilidad de CASE

Spark devuelve:

probability = [P(0), P(1)]

Necesitamos:

score = P(label = 1)

`sdf_separate_column()` puede separar directamente el vector Spark en columnas escalares. [Posit](#)

```
add_score <- function(predictions) {
```

```
  predictions %>%
```

```
    sdf_separate_column(
      column = "probability",
      into = list(
        probability_0 = 1,
        score = 2
      )
    )
  )
```

```
}
```

## 8. AUPRC y AUROC

Spark puede evaluar directamente ambos desde `rawPrediction`. AUPRC se solicita con `metric_name = "areaUnderPR"` y AUROC con `"areaUnderROC"`

```
get_auc_metrics <- function(predictions) {

  auprc <- ml_binary_classification_evaluator(
    predictions,
    label_col = "label",
    raw_prediction_col = "rawPrediction",
    metric_name = "areaUnderPR"
  )

  auroc <- ml_binary_classification_evaluator(
    predictions,
    label_col = "label",
    raw_prediction_col = "rawPrediction",
    metric_name = "areaUnderROC"
  )

  return(
    list(
      AUPRC = auprc,
      AUROC = auroc
    )
  )
}
```

## 9. Función para Top 1%, 5% y 10%

```
top_metrics <- function(
  scored_predictions,
  top_pct
) {

  # =====
  # SCORE THRESHOLD
  #
  # top 1%  -> P99
  # top 5%  -> P95
  # top 10% -> P90
```

```

# =====

quantile_prob <- 1 - top_pct

threshold <- sdf_quantile(
  scored_predictions,
  column = "score",
  probabilities = quantile_prob,
  relative.error = 0.0001
)

threshold <- as.numeric(threshold[1])

# =====
# FULL VALIDATION POPULATION
# =====

total_stats <- scored_predictions %>%

  summarise(
    n = n(),
    cases = sum(label),
    base_rate = mean(label)
  ) %>%

  collect()

# =====
# TOP-K POPULATION
# =====

selected <- scored_predictions %>%
  filter(score >= threshold)

selected_stats <- selected %>%

  summarise(
    selected_n = n(),

```

```

        selected_cases = sum(label),
        positive_rate = mean(label)
    ) %>%

    collect()

# =====
# METRICS
# =====

precision <- selected_stats$positive_rate

recall <- (
    selected_stats$selected_cases /
    total_stats$cases
)

lift <- (
    selected_stats$positive_rate /
    total_stats$base_rate
)

actual_selected_pct <- (
    selected_stats$selected_n /
    total_stats$n
)

return(
    list(
        requested_top_pct = top_pct,
        actual_selected_pct = actual_selected_pct,
        threshold = threshold,
        precision = precision,
        recall = recall,
        lift = lift
    )
)
}

```

10. Función de evaluación completa

```

evaluate_model <- function(predictions) {

  # =====
  # AUPRC / AUROC
  # =====

  auc_metrics <- get_auc_metrics(
    predictions
  )

  # =====
  # EXTRACT SCORE
  # =====

  scored <- add_score(
    predictions
  )

  # =====
  # TOP-K
  # =====

  top1 <- top_metrics(
    scored,
    0.01
  )

  top5 <- top_metrics(
    scored,
    0.05
  )

  top10 <- top_metrics(
    scored,
    0.10
  )

```

```

# =====
# OUTPUT
# =====

return(
  list(

    AUPRC = auc_metrics$AUPRC,
    AUROC = auc_metrics$AUROC,

    Lift_1 = top1$lift,
    Precision_1 = top1$precision,
    Recall_1 = top1$recall,
    ActualPct_1 = top1$actual_selected_pct,

    Lift_5 = top5$lift,
    Precision_5 = top5$precision,
    Recall_5 = top5$recall,
    ActualPct_5 = top5$actual_selected_pct,

    Lift_10 = top10$lift,
    Precision_10 = top10$precision,
    Recall_10 = top10$recall,
    ActualPct_10 = top10$actual_selected_pct
  )
)
}

```

## 1. Ejecutar Model A y Model B

```

results <- list()

result_index <- 1

for (reg in reg_params) {

```

```

cat(
  "\n===== \n"
)

cat(
  "Running regParam =",
  reg,
  "\n"
)

cat(
  "===== \n"
)

# =====
# MODEL A
# =====

cat(
  "\nFitting MODEL A... \n"
)

fit_A <- fit_logistic_model(
  train_data = train_df,
  validation_data = validation_df,
  formula = formula_A,
  reg_param = reg
)

metrics_A <- evaluate_model(
  fit_A$predictions
)

results[[result_index]] <- tibble(

  model = "A_CURRENT",
  regParam = reg,

```

```

    AUPRC = metrics_A$AUPRC,
    AUROC = metrics_A$AUROC,

    Lift_1 = metrics_A$Lift_1,
    Precision_1 = metrics_A$Precision_1,
    Recall_1 = metrics_A$Recall_1,

    Lift_5 = metrics_A$Lift_5,
    Precision_5 = metrics_A$Precision_5,
    Recall_5 = metrics_A$Recall_5,

    Lift_10 = metrics_A$Lift_10,
    Precision_10 = metrics_A$Precision_10,
    Recall_10 = metrics_A$Recall_10
  )

  result_index <- result_index + 1

  # =====
  # MODEL B
  # =====

  cat(
    "\nFitting MODEL B...\n"
  )

  fit_B <- fit_logistic_model(
    train_data = train_df,
    validation_data = validation_df,
    formula = formula_B,
    reg_param = reg
  )

  metrics_B <- evaluate_model(
    fit_B$predictions
  )

```

```

results[[result_index]] <- tibble(

  model = "B_CURRENT_PLUS_Z",
  regParam = reg,

  AUPRC = metrics_B$AUPRC,
  AUROC = metrics_B$AUROC,

  Lift_1 = metrics_B$Lift_1,
  Precision_1 = metrics_B$Precision_1,
  Recall_1 = metrics_B$Recall_1,

  Lift_5 = metrics_B$Lift_5,
  Precision_5 = metrics_B$Precision_5,
  Recall_5 = metrics_B$Recall_5,

  Lift_10 = metrics_B$Lift_10,
  Precision_10 = metrics_B$Precision_10,
  Recall_10 = metrics_B$Recall_10
)

result_index <- result_index + 1
}

```

## 12. Consolidar los resultados

```

results_df <- bind_rows(
  results
)

```

```
results_df
```

```

results_df %>%
  arrange(
    desc(AUPRC)
  )

```

```
results_df %>%
```

```

select(
  model,
  regParam,

  AUPRC,
  AUROC,

  Lift_1,
  Lift_5,
  Lift_10,

  Precision_1,
  Precision_5,
  Precision_10,

  Recall_1,
  Recall_5,
  Recall_10
) %>%

arrange(
  model,
  regParam
)

```

## Hay una cosa que quiero que hagamos antes de ejecutar las 10 corridas

Corre primero solamente **una prueba** con:

```

Model A
regParam = 0.01

```

usando:

```

test_fit <- fit_logistic_model(
  train_data = train_df,
  validation_data = validation_df,
  formula = formula_A,
  reg_param = 0.01
)

```

```
)  
  
test_metrics <- evaluate_model(  
  test_fit$predictions  
)  
  
test_metrics
```